



Sistemas embebidos

Laboratorio 3 – FreeRTOS

Integrantes del grupo:

- Agustina Bacigalupe
 - Pierina Borsieri
 - Sofia Nicoletti
 - Adrian Tesore

Docente: Pablo Alonso

Docente asistente: Nicolas Calarco

Fecha de entrega:

17 de junio de 2025

Objetivo:

Utilizar el sistema operativo en tiempo real FreeRTOS para trabajar con una arquitectura de software de multitasking tomando como base los requerimientos del laboratorio 2. Aplicar las funcionalidades FreeRTOS, timers, memoria dinámica, queues y semáforos.

Crear tres tareas, llamadas task A, task B, task C con diferentes funciones.

Task A:

En la siguiente task se buscó hacer parpadear el led, tomando el color como parámetro de entrada.

Para lograr esto se utilizaron las librerías del laboratorio 1, llamadas led y led strip ya que se buscaba una función similar. En cuanto a la elección del color que será mostrado en el led de la placa, se realizó un switch case dentro de un while(1) que en base a la variable llamada color, setea el color elegido tomando en cuenta diferentes parámetros.

Por último, utilizamos el delay de FreeRTOS, para luego poder apagar.

Task B:

En este caso se nos pidió crear una segunda task, encargada de gestionar el puerto UART, cumpliendo la función de recibir comandos y colocarlos en una queue para poder ser ejecutados. Para poder realizar dicha acción, se tuvo que investigar sobre cómo es el protocolo de comunicación UART. A la hora de esto, se tomó un ejemplo brindado por la página del ESP32, en específico el código uart_events, donde se realizaba una implementación similar.

Luego de ejecutar y entender cómo funcionaba dicho ejemplo, se procedió a implementar las funciones que serían útiles en nuestro caso dentro de la task_b de nuestro programa. En específico, se utilizó la representación de la lógica de recepción de eventos y leído de datos de UART. La task_b monitorea la uart_queue (una cola de eventos UART preconfigurada) para detectar la llegada de nuevos datos. Cuando se recibe un evento de tipo UART_DATA, se procede a leer los bytes disponibles del puerto UART.

Para permitir la recepción de comandos largos o múltiples comandos enviados en una única secuencia de texto, se implementó un buffer de entrada (input_buffer). Los caracteres leídos del UART se van acumulando en este buffer. Se definió un carácter de terminación, en este caso el punto "." como el indicador de que un comando ha sido recibido. Al detectar este carácter, la cadena acumulada en input_buffer se considera completa, se termina con un carácter nulo (\0) y es entonces cuando se pasa como parámetro a la función parse_and_enqueue_commands.

Una vez procesado el comando, el `input_buffer` se reinicia para recibir la siguiente secuencia. Se incorporó además una validación para evitar el desbordamiento de este `input_buffer`, descartando la entrada y alertando con un `ESP_LOGW` si el tamaño excede el límite predefinido. Asimismo, se gestionan eventos como `UART_FIFO_OVF` o `UART_BUFFER_FULL` mediante el vaciado del buffer de entrada del UART y el reseteo de la `uart_queue`, asegurando la recuperación del sistema ante posibles desbordamientos.

Continuando, la mencionada función `parse_and_enqueue_commands` recibe esta cadena completa (del tipo "rojo-100" o "rojo-100,verde-200,azul-50"), separando de esta manera en color y tiempo, para luego convertirlo en una estructura y así poder mandarlo a la queue utilizando `xQueueSend`. En nuestro caso usamos los colores rojo, verde y azul y el tiempo en ms.

Para soportar el envío de múltiples comandos en una única cadena de entrada, como por ejemplo "rojo-100,verde-200,azul-50", se mejoró la función `parse_and_enqueue_commands`. Ahora, esta función es capaz de dividir la cadena recibida en tokens individuales utilizando el delimitador de coma (,) mediante la función `strtok`. Cada token ("rojo-100", "verde-200", etc.) es procesado de forma independiente, extrayendo su color y tiempo correspondientes.

Debido a que el envío simultáneo de varios comandos podía saturar la `command_queue` y causar un bloqueo de la `task_b` (si `xQueueSend` esperaba por siempre), se implementó otro mecanismo de encolado. En el cual, cada comando individual se intenta encolar con un tiempo limitado (50 ms). Si la cola está llena, la `task_b` no se bloquea, sino que reintenta el envío hasta un máximo de 5 veces (`MAX_RETRY_ENQUEUE`), con una pequeña pausa entre cada reintento para dar tiempo a la tarea a liberar espacio. Si, al finalizar los reintentos, el comando no puede ser encolado, se registra un mensaje de error (`ESP_LOGE`) y el comando se descarta. En caso de éxito, se emite un mensaje informativo (`ESP_LOGI`).

Adicionalmente, para mejorar la capacidad del sistema de manejar los comandos, se aumentó el tamaño de la `command_queue` (definido por `QUEUE_LENGTH`) a uno valor que permitiera almacenar una mayor cantidad de elementos en espera.

```
static void parse_and_enqueue_commands(const char *input) {
    char buffer[BUF_SIZE];
    strncpy(buffer, input, BUF_SIZE - 1);
    buffer[BUF_SIZE - 1] = '\0';

    char *token = strtok(buffer, ",");
    while (token != NULL) {
        char *dash = strchr(token, '-');
        if (dash != NULL) {
            *dash = '\0';
            char *color = token;
            char *time_str = dash + 1;

            color_command_t cmd;
            strncpy(cmd.color, color, sizeof(cmd.color) - 1);
            cmd.color[sizeof(cmd.color) - 1] = '\0';
            cmd.tiempo_ms = atoi(time_str);

            if (xQueueSend(command_queue, &cmd, pdMS_TO_TICKS(100)) != pdPASS) {
                ESP_LOGW(TAG, "Cola llena, comando descartado: %s-%s", color, time_str);
            } else {
                ESP_LOGI(TAG, "Encolado: %s - %dms", cmd.color, cmd.tiempo_ms);
            }
        } else {
            ESP_LOGW(TAG, "Formato inválido: %s", token);
        }
        token = strtok(NULL, ",");
    }
}
```

Código de parseo y encolado de los comandos ingresados por consola y enviados por UART

```
verde=500,rojo=5000,verde=2000,rojo=500,azul=1000,verde=500,rojo=5000,verde=2000,rojo=500,azul=1000,verd
e=500,rojo=5000,verde=2000,rojo=500,azul=1000,verde=500,rojo=5000,verde=2000,rojo=500,azul=1000,verde=50
0,rojo=5000,verde=2000
I (11626) TaskB: Encolado: rojo - 500ms
I (11636) TaskB: Encolado: azul - 1000ms
I (11636) TaskB: Encolado: verde - 500ms
I (11646) TaskB: Encolado: rojo - 5000ms
I (11646) TaskB: Encolado: verde - 2000ms
I (11646) TaskB: Encolado: rojo - 500ms
I (11656) TaskB: Encolado: azul - 1000ms
I (11656) TaskB: Encolado: verde - 500ms
I (11656) TaskB: Encolado: rojo - 5000ms
I (11666) TaskB: Encolado: verde - 2000ms
I (11666) TaskB: Encolado: rojo - 500ms
Task C: Recibido color=rojo (1), duración=500ms
I (11676) TaskB: Encolado: azul - 1000ms
Task C: Recibido color=azul (3), duración=1000ms
I (11686) TaskB: Encolado: verde - 500ms
Task C: Recibido color=verde (2), duración=500ms
I (11686) TaskB: Encolado: rojo - 5000ms
Task C: Recibido color=rojo (1), duración=500ms
I (11696) TaskB: Encolado: verde - 2000ms
Task C: Recibido color=verde (2), duración=2000ms
I (11706) TaskB: Encolado: rojo - 500ms
Task C: Recibido color=rojo (1), duración=500ms
I (11716) TaskB: Encolado: azul - 1000ms
Task C: Recibido color=azul (3), duración=1000ms
I (11726) TaskB: Encolado: verde - 500ms
Task C: Recibido color=verde (2), duración=500ms
I (11736) TaskB: Encolado: rojo - 5000ms
Task C: Recibido color=rojo (1), duración=500ms
I (11736) TaskB: Encolado: verde - 2000ms
Task C: Recibido color=verde (2), duración=2000ms
I (11746) TaskB: Encolado: rojo - 500ms
Task C: Recibido color=rojo (1), duración=500ms
I (11756) TaskB: Encolado: azul - 1000ms
Task C: Recibido color=azul (3), duración=1000ms
I (11766) TaskB: Encolado: verde - 500ms
Task C: Recibido color=verde (2), duración=500ms
I (11776) TaskB: Encolado: rojo - 5000ms
Task C: Recibido color=rojo (1), duración=500ms
I (11776) TaskB: Encolado: verde - 2000ms
Task C: Recibido color=verde (2), duración=2000ms
I (11786) TaskB: Encolado: rojo - 500ms
Task C: Recibido color=rojo (1), duración=500ms
I (11796) TaskB: Encolado: azul - 1000ms
Task C: Recibido color=azul (3), duración=1000ms
I (11806) TaskB: Encolado: verde - 500ms
Task C: Recibido color=verde (2), duración=500ms
I (11816) TaskB: Encolado: rojo - 5000ms
Task C: Recibido color=rojo (1), duración=500ms
I (11816) TaskB: Encolado: verde - 2000ms
Task C: Recibido color=verde (2), duración=2000ms
I (11826) TaskB: Encolado: rojo - 500ms
Task C: Recibido color=rojo (1), duración=500ms
```

Encolado múltiple de comandos

Se puede ver cómo se llena la queue y al liberar espacio, la task_b mete en la cola nuevamente nuevos comandos sin tener pérdidas de datos.

Task C:

Por último, se creó la task_c esta tarea es la encargada de recibir y ejecutar los comandos de cambio de color enviados desde la queue compartida con la task_b.

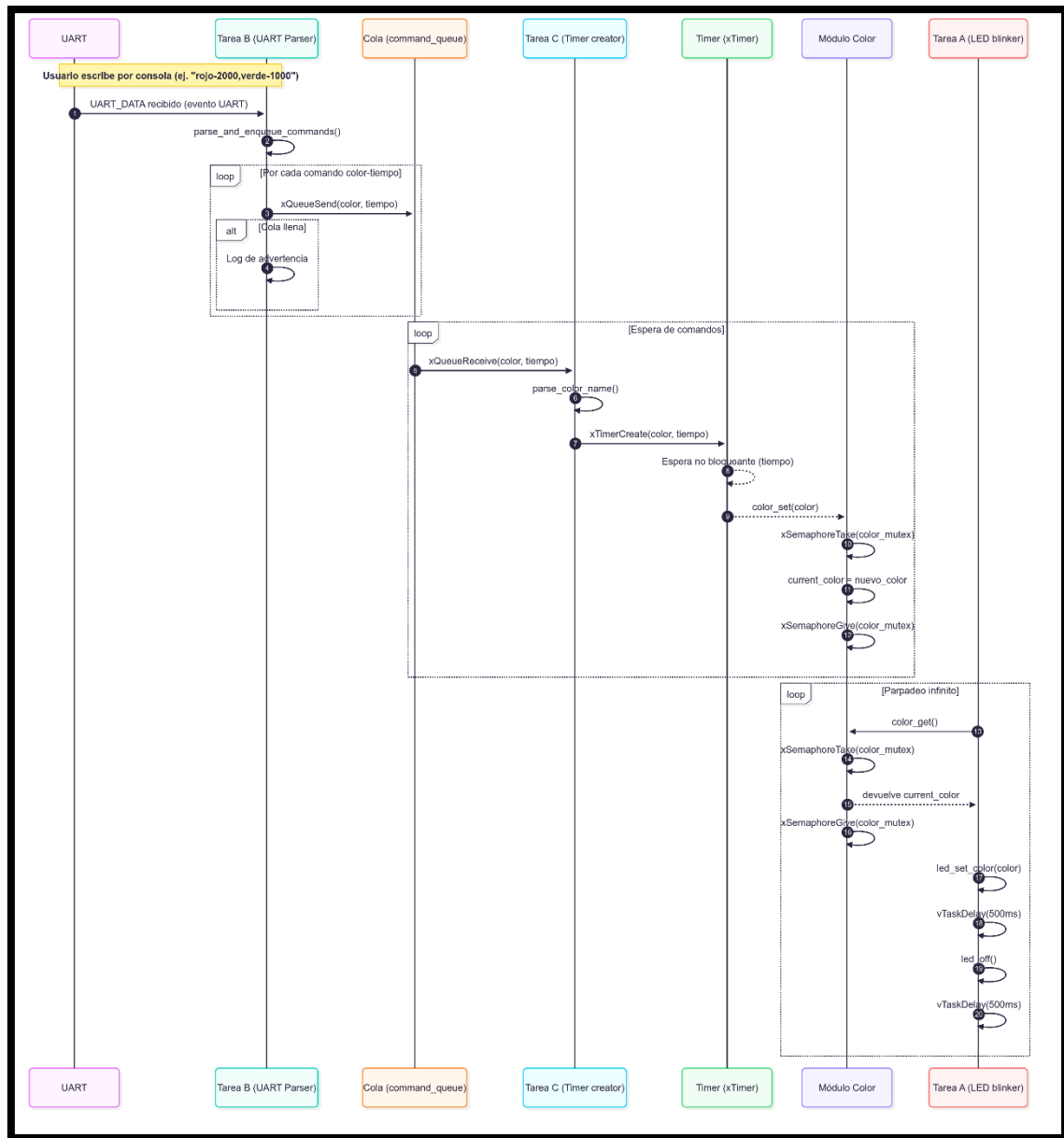
Como anteriormente se mencionó, los comandos contienen el nombre del color n y el tiempo. Dentro de la tarea, se utiliza la función xQueueReceive para recibir estructuras del tipo color_command_t. Una vez recibido el comando, se interpreta el color utilizando la función parse_color_name, que convierte cadenas como "rojo", "verde" o "azul" al tipo enumerado color_t.

Luego se crea un timer el cual espera el tiempo determinado en el par color-tiempo dentro de la queue. Al finalizar este timer se ejecuta la función color_timeout_callback(), la cual es la encargada de establecer el nuevo color del led, mediante la función color_set.

```
26 static void color_timeout_callback(TimerHandle_t xTimer) {
27     timer_data_t *data = (timer_data_t *)pvTimerGetTimerID(xTimer);
28     if (data != NULL) {
29         printf("Timer vencido, seteando color %d\n", data->color);
30         color_set(data->color);
31         free(data); // liberar memoria
32     }
33 }
34
35 /**
36  * @brief Convierte el nombre del color (string) al tipo `color_t`.
37  *
38  * @param color_name Cadena con el nombre del color ("rojo", "verde", "azul").
39  * @return color_t Valor correspondiente del enum o COLOR_NONE si no coincide.
40  */
41 static color_t parse_color_name(const char *color_name) {
42     if (strcmp(color_name, "rojo") == 0) return COLOR_RED;
43     if (strcmp(color_name, "verde") == 0) return COLOR_GREEN;
44     if (strcmp(color_name, "azul") == 0) return COLOR_BLUE;
45     return COLOR_NONE;
46 }
47
48 /**
```

Diagrama de secuencia UML:

El diagrama de secuencia presentado ilustra el flujo de ejecución entre las diferentes tareas del sistema implementado con FreeRTOS. Representa la interacción entre los siguientes componentes: UART, la tarea que parsea comandos (Task B), la cola de comandos (Queue command_queue), la tarea que ejecuta timers (Task C), los temporizadores (Timer xTimer), el módulo de control de color y la tarea encargada del parpadeo de LEDs (Task A).



Conclusión:

La implementación realizada en este laboratorio permitió reforzar los conocimientos sobre el sistema operativo en tiempo real FreeRTOS, aplicando los conceptos fundamentales como la creación y gestión de tareas concurrentes, la comunicación entre tareas mediante colas, manejo de timers y uso de memoria dinámica.

Mediante el desarrollo de las tareas A, B y C se logró construir un sistema funcional en el cual:

La Task A se encargó del control del LED mediante el uso de parámetros dinámicos para el color y el tiempo de encendido.

La Task B gestionó la recepción de comandos vía UART, su interpretación y posterior envío a una cola.

La Task C interpretó dichos comandos y aplicó los cambios correspondientes, demostrando el uso adecuado de timer para controlar la ejecución retardada de eventos.

En conclusión, en este trabajo se integraron múltiples componentes de FreeRTOS de manera coordinada, simulando una arquitectura típica de sistemas embebidos multitarea. Asimismo, se reforzó la capacidad de análisis, diseño de software, herramientas fundamentales para el desarrollo profesional en este campo.